

Part 1: Your Hypotheses

Before running any experiments, predict the rejection rate for each defense configuration against each attack type. Write a percentage (0% or 100%) in each cell.

Experiment	Immediate Replay	Delayed Replay	Modified Replay
No defense	0%	0%	0%
Timestamp only	0%	100%	0%
Counter only	100%	100%	0%
All defenses	100%	100%	100%

Part 2: Experiment Results

Experiment 1: No Defense (Baseline)

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	5	0	0%
Delayed replay	5	0	0%
Modified replay	5	0	0%

Experiment 2: Timestamp Only

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	5	0	0%
Delayed replay	0	5	100%
Modified replay	5	0	0%

Experiment 3: Counter Only

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	0	5	100%
Delayed replay	0	5	100%
Modified replay	5	0	0%

Experiment 4: All Defenses

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	0	5	100%
Delayed replay	0	5	100%
Modified replay	0	5	100%

Part 3: Summary Table

Compile all your results into a single table:

Defense	Immediate Replay	Delayed Replay	Modified Replay
No defense	0%	0%	0%
Timestamp only	0%	100%	0%
Counter only	100%	100%	0%
All defenses	100%	100%	100%

Part 4: Screenshots

Terminal output (experiment summary):

```
Last login: Fri May 1 02:12:04 on ttys003
kaylacollett@Mac hydroficient-project % python3 defense_tester.py --defense none --attack all

=====
EXPERIMENT: Defense=none vs Attack=immediate
=====

[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=50.99 LPM
Message 2: seq=2, flow=53.74 LPM
Message 3: seq=3, flow=46.83 LPM
Message 4: seq=4, flow=53.42 LPM
Message 5: seq=5, flow=52.97 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - No defenses active
[ACCEPTED] seq=2 - No defenses active
[ACCEPTED] seq=3 - No defenses active
[ACCEPTED] seq=4 - No defenses active
[ACCEPTED] seq=5 - No defenses active

[STEP 3] Creating immediate replay attack...
Replaying 5 messages (2s delay)

[STEP 4] Running attack...
[ACCEPTED] seq=1 - No defenses active
[ACCEPTED] seq=2 - No defenses active
[ACCEPTED] seq=3 - No defenses active
[ACCEPTED] seq=4 - No defenses active
[ACCEPTED] seq=5 - No defenses active

[RESULTS]
Messages tested: 5
Accepted: 5
Rejected: 0
Rejection rate: 0%

=====
EXPERIMENT: Defense=none vs Attack=delayed
=====

[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=53.5 LPM
Message 2: seq=2, flow=51.61 LPM
Message 3: seq=3, flow=52.83 LPM
Message 4: seq=4, flow=53.21 LPM
Message 5: seq=5, flow=49.38 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - No defenses active
[ACCEPTED] seq=2 - No defenses active
[ACCEPTED] seq=3 - No defenses active
[ACCEPTED] seq=4 - No defenses active
[ACCEPTED] seq=5 - No defenses active

[STEP 3] Creating delayed replay attack...
Replaying 5 messages (simulating 60s delay)

[STEP 4] Running attack...
[ACCEPTED] seq=1 - No defenses active
[ACCEPTED] seq=2 - No defenses active
[ACCEPTED] seq=3 - No defenses active
[ACCEPTED] seq=4 - No defenses active
[ACCEPTED] seq=5 - No defenses active
```

```
[RESULTS]
Messages tested: 5
Accepted: 5
Rejected: 0
Rejection rate: 0%

=====
EXPERIMENT: Defense=none vs Attack=modified
=====

[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=46.14 LPM
Message 2: seq=2, flow=45.35 LPM
Message 3: seq=3, flow=49.78 LPM
Message 4: seq=4, flow=45.17 LPM
Message 5: seq=5, flow=51.66 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - No defenses active
[ACCEPTED] seq=2 - No defenses active
[ACCEPTED] seq=3 - No defenses active
[ACCEPTED] seq=4 - No defenses active
[ACCEPTED] seq=5 - No defenses active

[STEP 3] Creating modified replay attack...
Replaying 5 MODIFIED messages

[STEP 4] Running attack...
[ACCEPTED] seq=6 - No defenses active
[ACCEPTED] seq=7 - No defenses active
[ACCEPTED] seq=8 - No defenses active
[ACCEPTED] seq=9 - No defenses active
[ACCEPTED] seq=10 - No defenses active

[RESULTS]
Messages tested: 5
Accepted: 5
Rejected: 0
Rejection rate: 0%

=====
EXPERIMENT SUMMARY
=====
Defense    Attack    Accepted    Rejected    Rate
-----
none       immediate 5           0           0%
none       delayed   5           0           0%
none       modified  5           0           0%

[SAVED] Results saved to experiment_results.json
kaylacolletti@Mac hydrofluent-project % python3 defense_tester.py --defense timestamp --attack all
[
=====
EXPERIMENT: Defense=timestamp vs Attack=immediate
=====

[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=50.67 LPM
Message 2: seq=2, flow=54.94 LPM
Message 3: seq=3, flow=50.81 LPM
Message 4: seq=4, flow=51.77 LPM
Message 5: seq=5, flow=50.67 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - All checks passed
[ACCEPTED] seq=2 - All checks passed
[ACCEPTED] seq=3 - All checks passed
[ACCEPTED] seq=4 - All checks passed
[ACCEPTED] seq=5 - All checks passed

[STEP 3] Creating immediate replay attack...
Replaying 5 messages (2s delay)

[STEP 4] Running attack...
[ACCEPTED] seq=1 - All checks passed
[ACCEPTED] seq=2 - All checks passed
[ACCEPTED] seq=3 - All checks passed
[ACCEPTED] seq=4 - All checks passed
[ACCEPTED] seq=5 - All checks passed

[RESULTS]
Messages tested: 5
Accepted: 5
Rejected: 0
Rejection rate: 0%

=====
EXPERIMENT: Defense=timestamp vs Attack=delayed
=====

[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=48.63 LPM
Message 2: seq=2, flow=52.68 LPM
Message 3: seq=3, flow=45.81 LPM
Message 4: seq=4, flow=47.11 LPM
Message 5: seq=5, flow=51.3 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - All checks passed
[ACCEPTED] seq=2 - All checks passed
[ACCEPTED] seq=3 - All checks passed
[ACCEPTED] seq=4 - All checks passed
[ACCEPTED] seq=5 - All checks passed

[STEP 3] Creating delayed replay attack...
Replaying 5 messages (simulating 60s delay)

[STEP 4] Running attack...
[REJECTED] seq=1 - Message too old (60.0s > 30s)
[REJECTED] seq=2 - Message too old (60.0s > 30s)
[REJECTED] seq=3 - Message too old (60.0s > 30s)
[REJECTED] seq=4 - Message too old (60.0s > 30s)
[REJECTED] seq=5 - Message too old (60.0s > 30s)

[RESULTS]
Messages tested: 5
Accepted: 0
Rejected: 5
Rejection rate: 100%

=====
EXPERIMENT: Defense=timestamp vs Attack=modified
=====

[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=45.74 LPM
```



```
=====
EXPERIMENT: Defense=timestamp vs Attack=modified
=====

[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=46.74 LPM
Message 2: seq=2, flow=48.6 LPM
Message 3: seq=3, flow=52.52 LPM
Message 4: seq=4, flow=48.56 LPM
Message 5: seq=5, flow=49.51 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - All checks passed
[ACCEPTED] seq=2 - All checks passed
[ACCEPTED] seq=3 - All checks passed
[ACCEPTED] seq=4 - All checks passed
[ACCEPTED] seq=5 - All checks passed

[STEP 3] Creating modified replay attack...
Replaying 5 MODIFIED messages

[STEP 4] Running attack...
[ACCEPTED] seq=6 - All checks passed
[ACCEPTED] seq=7 - All checks passed
[ACCEPTED] seq=8 - All checks passed
[ACCEPTED] seq=9 - All checks passed
[ACCEPTED] seq=10 - All checks passed

[RESULTS]
Messages tested: 5
Accepted: 5
Rejected: 0
Rejection rate: 0%

=====
EXPERIMENT SUMMARY
=====


| Defense   | Attack    | Accepted | Rejected | Rate |
|-----------|-----------|----------|----------|------|
| timestamp | immediate | 5        | 0        | 0%   |
| timestamp | delayed   | 0        | 5        | 100% |
| timestamp | modified  | 5        | 0        | 0%   |


[SAVED] Results saved to experiment_results.json
kaylacolletti@Mac hydroficient-project % python3 defense_tester.py --defense counter --attack all
```

```
=====
EXPERIMENT: Defense=counter vs Attack=immediate
=====

[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=53.91 LPM
Message 2: seq=2, flow=45.3 LPM
Message 3: seq=3, flow=53.47 LPM
Message 4: seq=4, flow=50.63 LPM
Message 5: seq=5, flow=52.87 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - All checks passed
[ACCEPTED] seq=2 - All checks passed
[ACCEPTED] seq=3 - All checks passed
[ACCEPTED] seq=4 - All checks passed
[ACCEPTED] seq=5 - All checks passed
```

```
[STEP 3] Creating immediate replay attack...
Replaying 5 messages (2s delay)

[STEP 4] Running attack...
[REJECTED] seq=1 - Sequence 1 <= last seen 5
[REJECTED] seq=2 - Sequence 2 <= last seen 5
[REJECTED] seq=3 - Sequence 3 <= last seen 5
[REJECTED] seq=4 - Sequence 4 <= last seen 5
[REJECTED] seq=5 - Sequence 5 <= last seen 5

[RESULTS]
Messages tested: 5
Accepted: 0
Rejected: 5
Rejection rate: 100%
```

```
=====
EXPERIMENT: Defense=counter vs Attack=delayed
=====

[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=51.26 LPM
Message 2: seq=2, flow=54.63 LPM
Message 3: seq=3, flow=47.36 LPM
Message 4: seq=4, flow=50.18 LPM
Message 5: seq=5, flow=46.18 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - All checks passed
[ACCEPTED] seq=2 - All checks passed
[ACCEPTED] seq=3 - All checks passed
[ACCEPTED] seq=4 - All checks passed
[ACCEPTED] seq=5 - All checks passed

[STEP 3] Creating delayed replay attack...
Replaying 5 messages (simulating 60s delay)

[STEP 4] Running attack...
[REJECTED] seq=1 - Sequence 1 <= last seen 5
[REJECTED] seq=2 - Sequence 2 <= last seen 5
[REJECTED] seq=3 - Sequence 3 <= last seen 5
[REJECTED] seq=4 - Sequence 4 <= last seen 5
[REJECTED] seq=5 - Sequence 5 <= last seen 5

[RESULTS]
Messages tested: 5
Accepted: 0
Rejected: 5
Rejection rate: 100%
```

```
=====
EXPERIMENT: Defense=counter vs Attack=modified
=====

[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=52.99 LPM
Message 2: seq=2, flow=46.61 LPM
Message 3: seq=3, flow=51.99 LPM
Message 4: seq=4, flow=50.92 LPM
Message 5: seq=5, flow=49.5 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - All checks passed
```

```
[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - All checks passed
[ACCEPTED] seq=2 - All checks passed
[ACCEPTED] seq=3 - All checks passed
[ACCEPTED] seq=4 - All checks passed
[ACCEPTED] seq=5 - All checks passed

[STEP 3] Creating modified replay attack...
Replaying 5 MODIFIED messages

[STEP 4] Running attack...
[ACCEPTED] seq=6 - All checks passed
[ACCEPTED] seq=7 - All checks passed
[ACCEPTED] seq=8 - All checks passed
[ACCEPTED] seq=9 - All checks passed
[ACCEPTED] seq=10 - All checks passed

[RESULTS]
Messages tested: 5
Accepted: 5
Rejected: 0
Rejection rate: 0%
```

EXPERIMENT SUMMARY				
Defense	Attack	Accepted	Rejected	Rate
counter	immediate	0	5	100%
counter	delayed	0	5	100%
counter	modified	5	0	0%

```
[SAVED] Results saved to experiment_results.json
kaylacolletti@Mac hydroficient-project % python3 defense_tester.py --defense all --attack all
```

```
=====
EXPERIMENT: Defense=all vs Attack=immediate
=====
```

```
[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=54.57 LPM
Message 2: seq=2, flow=54.23 LPM
Message 3: seq=3, flow=49.11 LPM
Message 4: seq=4, flow=54.17 LPM
Message 5: seq=5, flow=54.0 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - All checks passed
[ACCEPTED] seq=2 - All checks passed
[ACCEPTED] seq=3 - All checks passed
[ACCEPTED] seq=4 - All checks passed
[ACCEPTED] seq=5 - All checks passed

[STEP 3] Creating immediate replay attack...
Replaying 5 messages (2s delay)

[STEP 4] Running attack...
[REJECTED] seq=1 - Sequence 1 <= last seen 5
[REJECTED] seq=2 - Sequence 2 <= last seen 5
[REJECTED] seq=3 - Sequence 3 <= last seen 5
[REJECTED] seq=4 - Sequence 4 <= last seen 5
[REJECTED] seq=5 - Sequence 5 <= last seen 5
```

```
[RESULTS]
Messages tested: 5
Accepted: 0
Rejected: 5
Rejection rate: 100%
```

```
=====
EXPERIMENT: Defense=all vs Attack=delayed
=====
```

```
[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=50.23 LPM
Message 2: seq=2, flow=49.91 LPM
Message 3: seq=3, flow=49.86 LPM
Message 4: seq=4, flow=52.8 LPM
Message 5: seq=5, flow=49.61 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - All checks passed
[ACCEPTED] seq=2 - All checks passed
[ACCEPTED] seq=3 - All checks passed
[ACCEPTED] seq=4 - All checks passed
[ACCEPTED] seq=5 - All checks passed

[STEP 3] Creating delayed replay attack...
Replaying 5 messages (simulating 60s delay)

[STEP 4] Running attack...
[REJECTED] seq=1 - Message too old (60.0s > 30s)
[REJECTED] seq=2 - Message too old (60.0s > 30s)
[REJECTED] seq=3 - Message too old (60.0s > 30s)
[REJECTED] seq=4 - Message too old (60.0s > 30s)
[REJECTED] seq=5 - Message too old (60.0s > 30s)
```

```
[RESULTS]
Messages tested: 5
Accepted: 0
Rejected: 5
Rejection rate: 100%
```

```
=====
EXPERIMENT: Defense=all vs Attack=modified
=====
```

```
[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=51.76 LPM
Message 2: seq=2, flow=67.88 LPM
Message 3: seq=3, flow=51.32 LPM
Message 4: seq=4, flow=48.3 LPM
Message 5: seq=5, flow=48.76 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - All checks passed
[ACCEPTED] seq=2 - All checks passed
[ACCEPTED] seq=3 - All checks passed
[ACCEPTED] seq=4 - All checks passed
[ACCEPTED] seq=5 - All checks passed
```

```
[STEP 3] Creating modified replay attack...
Replaying 5 MODIFIED messages
```

```
[STEP 4] Running attack...
[REJECTED] seq=6 - HMAC mismatch
[REJECTED] seq=7 - HMAC mismatch
```

```
[STEP 3] Creating delayed replay attack...
Replaying 5 messages (simulating 60s delay)

[STEP 4] Running attack...
[REJECTED] seq=1 - Message too old (60.0s > 30s)
[REJECTED] seq=2 - Message too old (60.0s > 30s)
[REJECTED] seq=3 - Message too old (60.0s > 30s)
[REJECTED] seq=4 - Message too old (60.0s > 30s)
[REJECTED] seq=5 - Message too old (60.0s > 30s)

[RESULTS]
Messages tested: 5
Accepted: 0
Rejected: 5
Rejection rate: 100%

=====
EXPERIMENT: Defense=all vs Attack=modified
=====

[STEP 1] Generating 5 legitimate messages...
Message 1: seq=1, flow=51.76 LPM
Message 2: seq=2, flow=47.58 LPM
Message 3: seq=3, flow=51.32 LPM
Message 4: seq=4, flow=48.3 LPM
Message 5: seq=5, flow=48.76 LPM

[STEP 2] Processing legitimate messages through subscriber...
[ACCEPTED] seq=1 - All checks passed
[ACCEPTED] seq=2 - All checks passed
[ACCEPTED] seq=3 - All checks passed
[ACCEPTED] seq=4 - All checks passed
[ACCEPTED] seq=5 - All checks passed

[STEP 3] Creating modified replay attack...
Replaying 5 MODIFIED messages

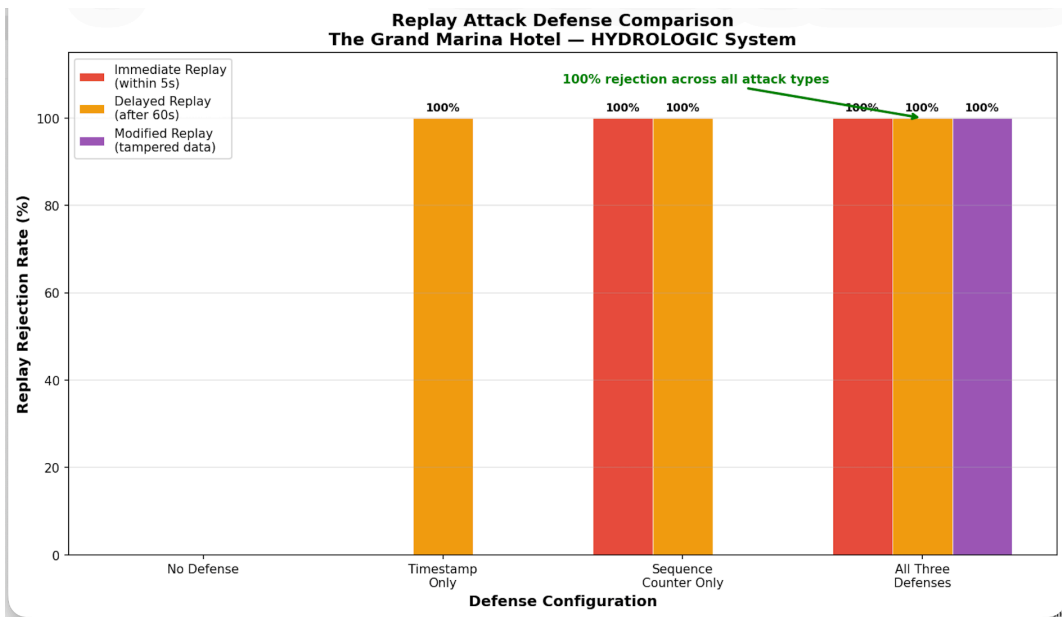
[STEP 4] Running attack...
[REJECTED] seq=6 - HMAC mismatch
[REJECTED] seq=7 - HMAC mismatch
[REJECTED] seq=8 - HMAC mismatch
[REJECTED] seq=9 - HMAC mismatch
[REJECTED] seq=10 - HMAC mismatch

[RESULTS]
Messages tested: 5
Accepted: 0
Rejected: 5
Rejection rate: 100%

=====
EXPERIMENT SUMMARY
=====
Defense    Attack    Accepted    Rejected    Rate
-----
all        immediate 0           5          100%
all        delayed   0           5          100%
all        modified  0           5          100%

[SAVED] Results saved to experiment_results.json
kaylacolletti@Mac hydroficient-project % python3 defense_tester.py --mode chart
[SAVED] Chart saved to defense_comparison.png
[INFO] Open defense_comparison.png to view the comparison
kaylacolletti@Mac hydroficient-project %
```

defense_comparison.png chart:



Part 5: Hypothesis Comparison

1. Which predictions were correct?

All of my predictions were correct. The rejection rates I expected matched the actual results for every defense and attack type. Each defense behaved the way I thought it would based on what it checks.

2. Which results surprised you? Were there any attack/defense combinations you expected to block that didn't, or vice versa?

The most surprising result was that modified replay attacks were not blocked by timestamp only or counter only defenses. I expected at least one of them to catch it, but it still accepted the messages. However once all defenses were combined, modified replay attacks were fully rejected, which shows that adding multiple layers fixed that gap.

3. Why did individual defenses have gaps? Think about what each defense checks and what it doesn't.

Each defense only checks one specific part of the message, so it misses other types of attacks. The timestamp defense only checks if the message is still fresh. It blocks delayed replay attacks because those are too old, but it does not block immediate replays since they are still within the time window. It also does not catch modified replays because it is not checking if the message content was changed, only the time. The counter only defense only checks if the counter value has already been used. It blocks both immediate and delayed replay attacks since those reuse the same counter. However it does not block modified replays because it is not verifying the actual message content, so a changed message can still get through. When all these defenses are used together, they cover each other's weaknesses, which is why all the attacks were rejected.